

Bayesian GAMs with Turing.jl

Simon Frost

Table of contents

Introduction	2
Setup	2
Example 1: Gaussian GAM	3
Data	3
Frequentist reference	3
Bayesian fit	3
Understanding the priors	4
Posterior summaries	4
Accessing posterior samples	5
Posterior density of <code>_obs</code>	5
Comparing posteriors: frequentist vs Bayesian	6
Posterior density of intercept [1]	6
Empirical CDF comparison	6
Posterior predictive check	7
Example 2: Poisson GAM	8
Data	8
Frequentist vs Bayesian	9
Posterior summary	9
ECDF comparison for intercept	9
Poisson fit with credible intervals	9
Cross-language comparison: GAM.jl vs brms vs mgcv	11
ECDF correlation helper	11
Gaussian model	12
ECDF comparison plots (Gaussian)	12
Poisson model	13
Example 3: Custom priors — effect on smoothing	13
Prior sensitivity: <code>_s</code> posterior	14
Example 4: Composable smooth terms with <code>smooth_prior</code>	14
Comparing <code>smooth_prior</code> to the brms-like <code>gam()</code> interface	16

Example 5: Low-level matrix extraction	19
Summary	20
Key design choices	20

Introduction

A **Bayesian GAM** replaces the penalized likelihood framework of a standard GAM with a fully probabilistic model. Instead of choosing smoothing parameters by REML or GCV, the Bayesian approach places **priors** on the smooth function variability and uses **MCMC** (Markov chain Monte Carlo) to obtain posterior distributions for all parameters.

The connection between penalized splines and Bayesian models is well established:

- A penalized spline $f(x) = \mathbf{Z}\mathbf{b}$ with penalty $\lambda\|\mathbf{b}\|^2$ is equivalent to a random effect $\mathbf{b} \sim N(0, \sigma_s^2 \mathbf{I})$ where $\sigma_s^2 = \sigma^2 / \lambda$
- The **smooth2random** decomposition splits each smooth into a fixed null-space part (unpenalized) and a random-effects part (penalized)
- Priors on σ_s control the amount of smoothing — this is the Bayesian analog of REML estimation of λ

GAM.jl implements this via a **Turing.jl package extension**: simply pass `priors = PriorSpec(...)` to `gam()` and the model is automatically reparameterized and sampled using NUTS (No-U-Turn Sampler).

Setup

```
using GAM
using Turing
using CSV
using DataFrames
using Distributions
using Statistics: mean, std, var, cor, median, quantile
using StatsAPI: coef, coeftable, confint, fitted, nobs
using Plots
using Printf
using LinearAlgebra: I
```

Example 1: Gaussian GAM

Data

Simulated data with $y = \sin(2\pi x) + \varepsilon$, $\varepsilon \sim N(0, 0.3^2)$.

n = 200, y range: [-1.48, 1.61]

Frequentist reference

```
m_freq = gam(@gam_formula(y ~ s(x, k = 10)), dat)
freq_int = coef(m_freq)[1]
freq_ = sqrt(m_freq.scale)
```

0.3133626090202822

Frequentist: intercept = -0.1129, = 0.3134, edf = 7.8

Bayesian fit

To trigger Bayesian fitting, pass a `PriorSpec` to `gam()`. The same formula syntax is used — the dispatch is automatic:

```
m_bayes = gam(@gam_formula(y ~ s(x, k = 10)), dat;
  priors = PriorSpec(sds = Exponential(1.0)),
  nsamples = 2000, nchains = 2)
```

Bayesian Generalized Additive Model

Formula: y ~ 1

Family: Normal

Link: IdentityLink

Sampler: Turing.Inference.NUTS{ADTypes.AutoForwardDiff{nothing, Nothing}, AdvancedHMC.DiagEu

Parametric coefficients:

	Estimate	Est.Error	1-95% CI	u-95% CI
--	----------	-----------	----------	----------

(Intercept)	-0.112891	0.0219965	-0.155981	-0.069571
s(x,bs=tp)_f1	-3.41417	1.0015	-5.33488	-1.38576

Smooth terms: s(x,bs=tp)
n = 200

Understanding the priors

PriorSpec controls the prior distributions:

- **sds**: Prior on σ_s , the SD of smooth random effects. An `Exponential(1.0)` prior is weakly informative, allowing the data to determine smoothness.
- **sigma**: Prior on σ_{obs} , the residual SD (Gaussian family only). Default: `truncated(Normal(0, 2.5); lower=0)`.
- **b**: Prior on fixed-effect coefficients. Default: `Normal(0, 10)`.

PriorSpec:

```
b (fixed effects):    Distributions.Normal{Float64}(=0.0, =10.0)
sds (smooth SDs):    Distributions.Exponential{Float64}(=1.0)
sigma (residual SD): Truncated(Distributions.Normal{Float64}(=0.0, =2.5); lower=0.0)
phi (dispersion):    Truncated(Distributions.Normal{Float64}(=0.0, =5.0); lower=0.0)
```

Posterior summaries

The `coefstable()` method returns posterior mean, SD, and 95% credible intervals:

	Estimate	Est.Error	l-95% CI	u-95% CI
(Intercept)	-0.112891	0.0219965	-0.155981	-0.069571
s(x,bs=tp)_f1	-3.41417	1.0015	-5.33488	-1.38576

Credible intervals at different levels:

Intercept CIs:

```
90%: [-0.1481, -0.0768]
95%: [-0.1560, -0.0696]
```

Accessing posterior samples

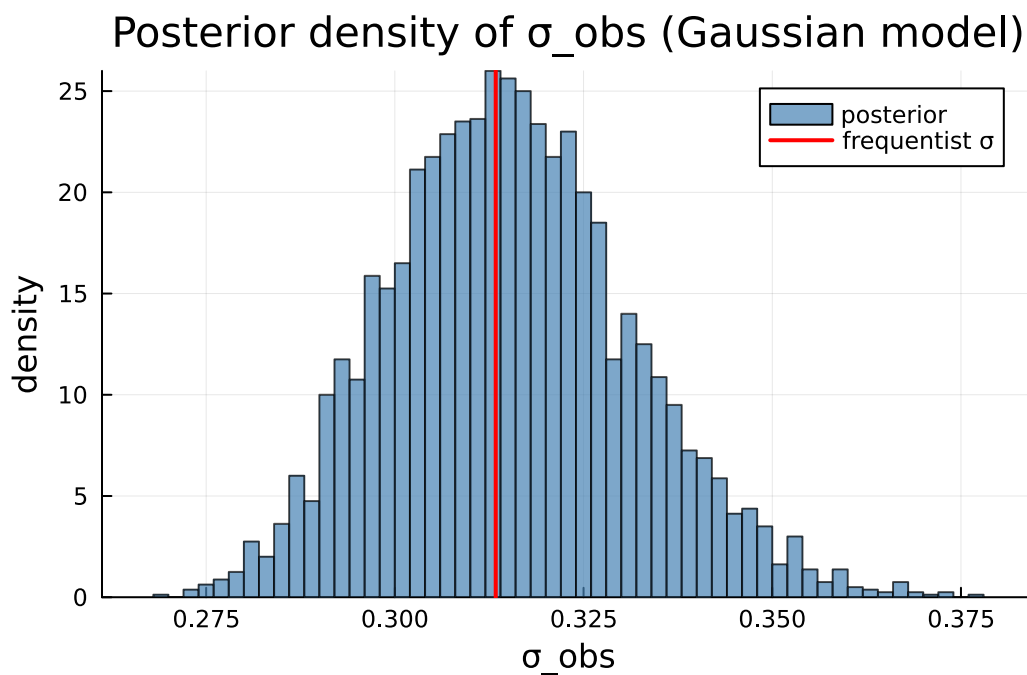
The full MCMC chains are accessible via `m_bayes.chains`:

```
_obs posterior: mean = 0.3154, sd = 0.0160, median = 0.3147
95% CI: [0.2864, 0.3489]
Frequentist : 0.3134

_s[1] posterior: mean = 1.6161, sd = 0.4276
Larger _s → more flexible smooth; smaller → smoother
```

Posterior density of `_obs`

```
histogram(_obs; normalize=:pdf, bins=50, alpha=0.7, color=:steelblue,
  label="posterior", xlabel="_obs", ylabel="density",
  title="Posterior density of _obs (Gaussian model)")
vline!([freq_]; color=:red, linewidth=2, label="frequentist ")
```

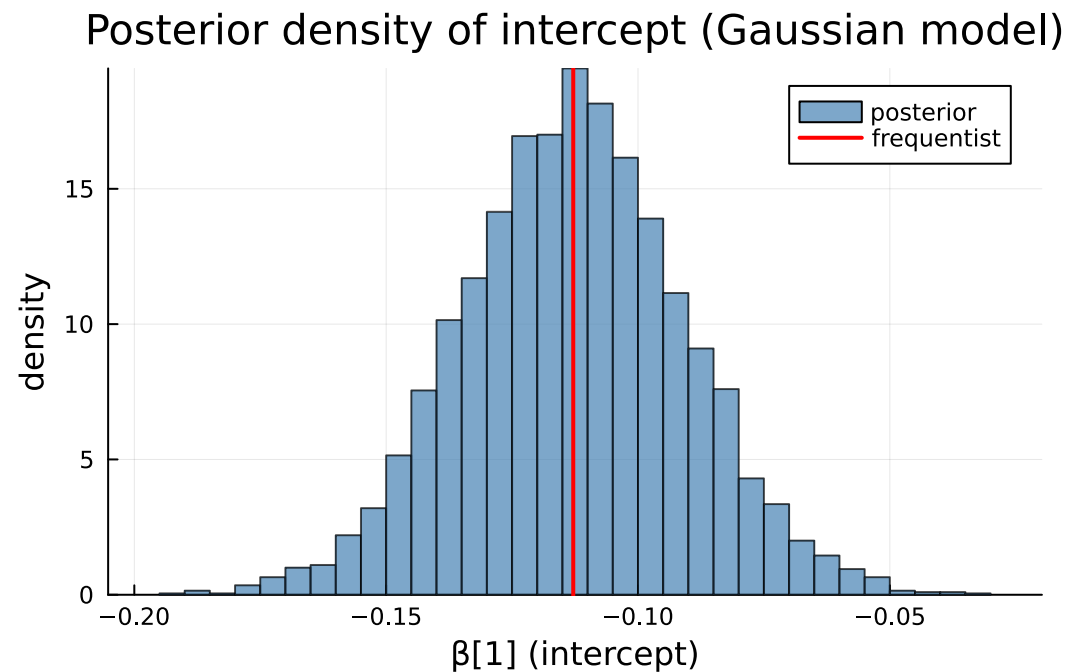


Comparing posteriors: frequentist vs Bayesian

Intercept: frequentist = -0.1129, Bayesian posterior mean = -0.1129
: frequentist = 0.3134, Bayesian posterior mean = 0.3154

Posterior density of intercept [1]

```
1_samples = vec(chains[Symbol(" [1]")].data)
histogram(1_samples; normalize=:pdf, bins=50, alpha=0.7, color=:steelblue,
          label="posterior", xlabel=" [1] (intercept)", ylabel="density",
          title="Posterior density of intercept (Gaussian model)")
vline!([freq_int]; color=:red, linewidth=2, label="frequentist")
```



Empirical CDF comparison

We compare the posterior distribution of σ_{obs} against the frequentist point estimate using the empirical CDF:

Frequentist = 0.3134 sits at 46.4% of the posterior ECDF
(values near 50% indicate good agreement)

```
ECDF 2.5%: _obs = 0.2863
ECDF 25.0%: _obs = 0.3042
ECDF 50.0%: _obs = 0.3147
ECDF 75.0%: _obs = 0.3255
ECDF 97.5%: _obs = 0.3489
```

Posterior predictive check

Compare the posterior fitted values (with 95% credible intervals) to the raw data and the frequentist fit:

```
# Reconstruct full design matrix
X_para, smooths, _ = GAM.gam_matrices(@gam_formula(y ~ s(x, k = 10)), dat)
Xf = smooths[1].Xf
Zs = smooths[1].Zs[1]

# Extract posterior draws for all parameters
chains = m_bayes.chains
n_draws = length(vec(chains[Symbol(" [1]")].data))
n_obs = nrow(dat)

# Compute fitted values for each posterior draw
_draws = Matrix{Float64}(undef, n_draws, n_obs)
for i in 1:n_draws
    1_i = chains[Symbol(" [1]")].data[i]
    2_i = chains[Symbol(" [2]")].data[i]
    _s_i = chains[Symbol(" _s[1]")].data[i]
    z_vec = [chains[Symbol("z[$j]")].data[i] for j in 1:size(Zs, 2)]
    _draws[i, :] = X_para * [1_i] .+ Xf * [2_i] .+ _s_i .* (Zs * z_vec)
end

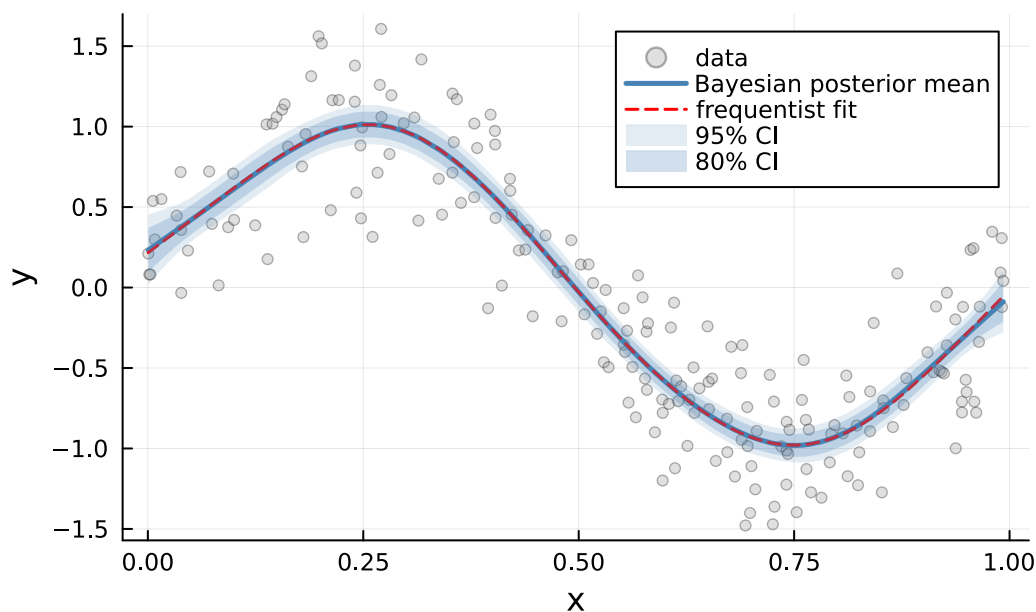
# Sort by x for plotting
order = sortperm(dat.x)
x_sorted = dat.x[order]

# Posterior summaries
_mean = vec(mean(_draws; dims=1))[order]
_lo = [quantile(_draws[:, j], 0.025) for j in 1:n_obs][order]
_hi = [quantile(_draws[:, j], 0.975) for j in 1:n_obs][order]
_lo80 = [quantile(_draws[:, j], 0.1) for j in 1:n_obs][order]
_hi80 = [quantile(_draws[:, j], 0.9) for j in 1:n_obs][order]
```

```
# Frequentist fitted
freq_fitted = fitted(m_freq)[order]

# Plot
scatter(dat.x, dat.y; alpha=0.3, color=:gray60, markersize=3, label="data",
        xlabel="x", ylabel="y", title="Gaussian GAM: data + Bayesian fit with credible intervals")
plot!(x_sorted, _mean; color=:steelblue, linewidth=2.5, label="Bayesian posterior mean")
plot!(x_sorted, freq_fitted; color=:red, linewidth=1.5, linestyle=:dash, label="frequentist")
plot!(x_sorted, _lo; fillrange=_hi, alpha=0.15, color=:steelblue, label="95% CI", linewidth=1)
plot!(x_sorted, _lo80; fillrange=_hi80, alpha=0.25, color=:steelblue, label="80% CI", linewidth=1)
```

Gaussian GAM: data + Bayesian fit with credible intervals



Example 2: Poisson GAM

Data

Count data with $\log(\lambda) = 1 + 1.5 \sin(2\pi x)$.

n = 200, y range: [0, 17]

Frequentist vs Bayesian

```
m_freq2 = gam(@gam_formula(y ~ s(x, k = 10)), dat2;
  family = Poisson(), link = LogLink())

m_bayes2 = gam(@gam_formula(y ~ s(x, k = 10)), dat2;
  family = Poisson(), link = LogLink(),
  priors = PriorSpec(sds = Exponential(1.0)),
  nsamples = 2000, nchains = 2)
```

Posterior summary

	Estimate	Est.Error	l-95% CI	u-95% CI
(Intercept)	0.908071	0.0548217	0.796993	1.01173
s(x,bs=tp)_f1	-3.65801	1.72698	-6.98944	-0.124977

Intercept (log-scale): frequentist = 0.9094, Bayesian = 0.9081 (true = 1.0)
_s[1]: mean = 2.2283, sd = 0.6150

ECDF comparison for intercept

Frequentist intercept = 0.9094 sits at 50.4% of Bayesian posterior
True intercept = 1.0 sits at 95.8% of posterior

Poisson fit with credible intervals

```
# Reconstruct design matrix for Poisson model
X_para2, smooths2, _ = GAM.gam_matrices(@gam_formula(y ~ s(x, k = 10)), dat2)
Xf2 = smooths2[1].Xf
Zs2 = smooths2[1].Zs[1]

n_draws2 = length(vec(chains2[Symbol(" [1]")].data))
n_obs2 = nrow(dat2)

# Compute linear predictor for each draw, then apply inverse link (exp)
```

```

_draws = Matrix{Float64}(undef, n_draws2, n_obs2)
for i in 1:n_draws2
    1_i = chains2[Symbol(" [1]")].data[i]
    2_i = chains2[Symbol(" [2]")].data[i]
    _s_i = chains2[Symbol("_s[1]")].data[i]
    z_vec = [chains2[Symbol("z[$j]")].data[i] for j in 1:size(Zs2, 2)]
    = X_para2 * [ 1_i] .+ Xf2 * [ 2_i] .+ _s_i .* (Zs2 * z_vec)
    _draws[i, :] = exp.() # inverse link for Poisson
end

order2 = sortperm(dat2.x)
x_sorted2 = dat2.x[order2]

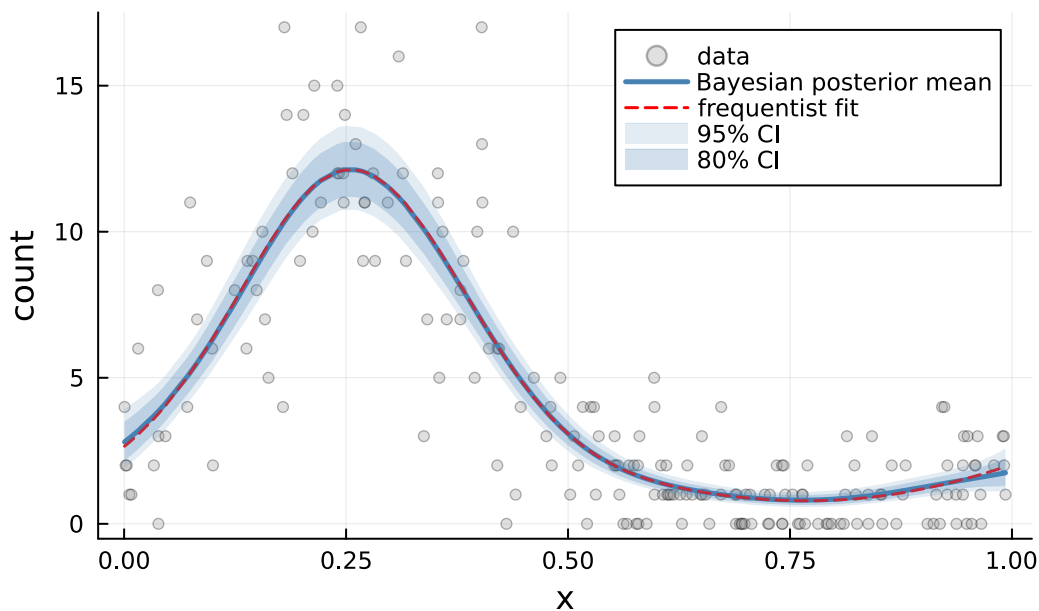
_mean = vec(mean(_draws; dims=1))[order2]
_lo = [quantile(_draws[:, j], 0.025) for j in 1:n_obs2][order2]
_hi = [quantile(_draws[:, j], 0.975) for j in 1:n_obs2][order2]
_lo80 = [quantile(_draws[:, j], 0.1) for j in 1:n_obs2][order2]
_hi80 = [quantile(_draws[:, j], 0.9) for j in 1:n_obs2][order2]

freq_fitted2 = fitted(m_freq2)[order2]

scatter(dat2.x, dat2.y; alpha=0.3, color=:gray60, markersize=3, label="data",
        xlabel="x", ylabel="count", title="Poisson GAM: data + Bayesian fit with credible intervals")
plot!(x_sorted2, _mean; color=:steelblue, linewidth=2.5, label="Bayesian posterior mean")
plot!(x_sorted2, freq_fitted2; color=:red, linewidth=1.5, linestyle=:dash, label="frequentist fit")
plot!(x_sorted2, _lo; fillrange=_hi, alpha=0.15, color=:steelblue, label="95% CI", linewidth=1.5)
plot!(x_sorted2, _lo80; fillrange=_hi80, alpha=0.25, color=:steelblue, label="80% CI", linewidth=1.5)

```

Poisson GAM: data + Bayesian fit with credible inter



Cross-language comparison: GAM.jl vs brms vs mgcv

We load posterior samples from R's brms (Stan MCMC) and mgcv (Gaussian approximation) to compare with GAM.jl's Turing posteriors. The key metric is the **pairwise ECDF correlation**: evaluate both ECDFs on a common grid and compute Pearson's r . A value of 1.0 means the distributions are identical.

Loaded: brms Gaussian=2000, brms Poisson=2000, mgcv Gaussian=4000, mgcv Poisson=4000

ECDF correlation helper

```
function ecdf_cor(x::AbstractVector, y::AbstractVector; n_grid::Int = 500)
    lo = min(minimum(x), minimum(y))
    hi = max(maximum(x), maximum(y))
    grid = range(lo, hi; length = n_grid)
    ecdf_x = [count(g, x) / length(x) for g in grid]
    ecdf_y = [count(g, y) / length(y) for g in grid]
    return cor(ecdf_x, ecdf_y)
end
```

ecdf_cor (generic function with 1 method)

Gaussian model

Parameter	GAM.jl (Turing)	brms (Stan)	mgcv (approx)
----- ----- ----- -----			
sigma mean	0.3154	0.3151	0.3145
sigma sd	0.0160	0.0161	0.0160
Intercept mean	-0.1129	-0.1131	-0.1123
Intercept sd	0.0220	0.0223	0.0221

Pairwise ECDF correlation (Gaussian model):

Comparison	sigma ECDF cor	intercept ECDF cor
----- ----- -----		
GAM.jl vs brms	0.999935	0.999944
GAM.jl vs mgcv (approx)	0.999838	0.999941
brms vs mgcv (approx)	0.999921	0.999904

ECDF comparison plots (Gaussian)

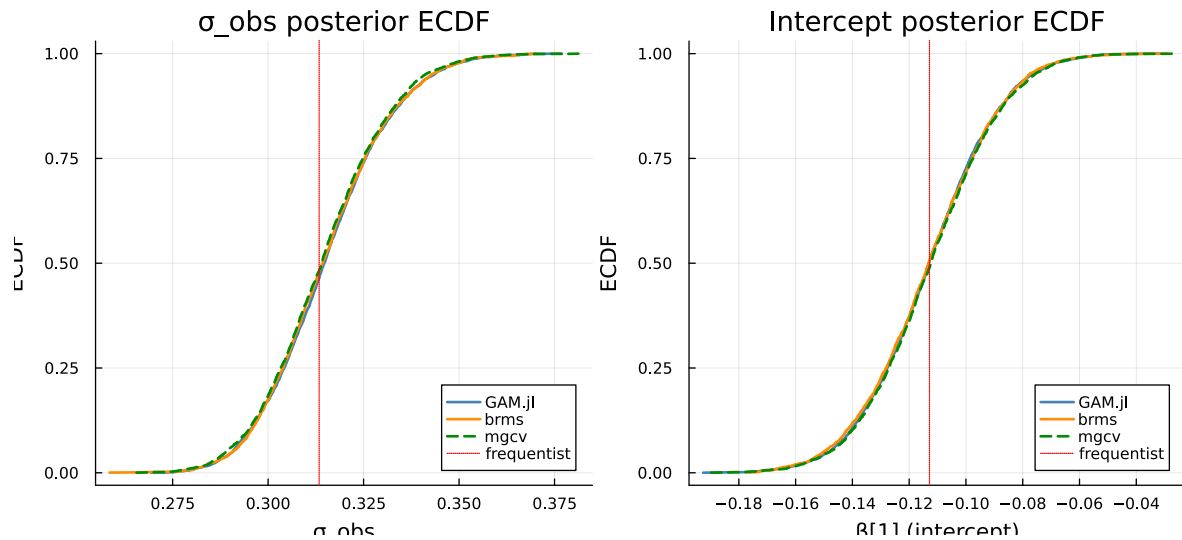
```

p1 = plot(sort(julia_sigma), (1:length(julia_sigma)) ./ length(julia_sigma);
        label="GAM.jl", color=:steelblue, linewidth=2,
        xlabel="_obs", ylabel="ECDF", title="_obs posterior ECDF")
plot!(p1, sort(brms_sigma), (1:length(brms_sigma)) ./ length(brms_sigma);
        label="brms", color=:darkorange, linewidth=2)
plot!(p1, sort(mgcv_sigma), (1:length(mgcv_sigma)) ./ length(mgcv_sigma);
        label="mgcv", color=:green4, linewidth=2, linestyle=:dash)
vline!(p1, [freq_]; color=:red, linewidth=1, linestyle=:dot, label="frequentist")

p2 = plot(sort(julia_int), (1:length(julia_int)) ./ length(julia_int);
        label="GAM.jl", color=:steelblue, linewidth=2,
        xlabel="[1] (intercept)", ylabel="ECDF", title="Intercept posterior ECDF")
plot!(p2, sort(brms_int), (1:length(brms_int)) ./ length(brms_int);
        label="brms", color=:darkorange, linewidth=2)
plot!(p2, sort(mgcv_int), (1:length(mgcv_int)) ./ length(mgcv_int);
        label="mgcv", color=:green4, linewidth=2, linestyle=:dash)
vline!(p2, [freq_int]; color=:red, linewidth=1, linestyle=:dot, label="frequentist")

plot(p1, p2; layout=(1, 2), size=(900, 400), legend=:bottomright)

```



Poisson model

Pairwise ECDF correlation (Poisson intercept):

GAM.jl vs brms: 0.999921

GAM.jl vs mgcv (approx): 0.999956

brms vs mgcv (approx): 0.999957

Example 3: Custom priors — effect on smoothing

The prior on `sds` directly controls the amount of smoothing. A tighter prior yields smoother fits:

```
# Tight prior: Exponential(0.1) → small _s → smoother
m_tight = gam(@gam_formula(y ~ s(x, k = 10)), dat;
  priors = PriorSpec(sds = Exponential(0.1)),
  nsamples = 1000, nchains = 1)

# Wide prior: Exponential(5.0) → large _s → wigglier
m_wide = gam(@gam_formula(y ~ s(x, k = 10)), dat;
  priors = PriorSpec(sds = Exponential(5.0)),
  nsamples = 1000, nchains = 1)
```

Tight prior (Exp(0.1)): posterior mean `_s` = 1.0002

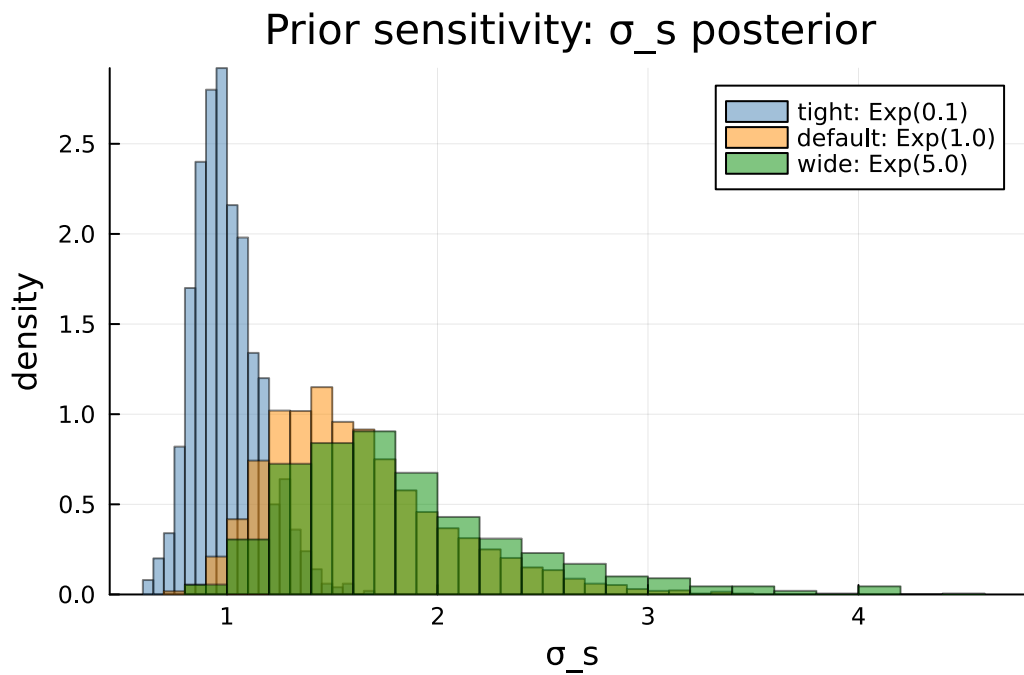
Wide prior (Exp(5.0)): posterior mean `_s` = 1.8472

Ratio: 1.8x

Prior sensitivity: σ_s posterior

```
_s_tight = vec(m_tight.chains[Symbol("_s[1]")].data)
_s_default = _s # from default Exp(1.0) model
_s_wide = vec(m_wide.chains[Symbol("_s[1]")].data)

histogram(_s_tight; normalize=:pdf, bins=30, alpha=0.5, color=:steelblue,
  label="tight: Exp(0.1)", xlabel="_s", ylabel="density",
  title="Prior sensitivity:  $\sigma_s$  posterior")
histogram!(_s_default; normalize=:pdf, bins=30, alpha=0.5, color=:darkorange,
  label="default: Exp(1.0)")
histogram!(_s_wide; normalize=:pdf, bins=30, alpha=0.5, color=:green4,
  label="wide: Exp(5.0)")
```



Example 4: Composable smooth terms with `smooth_prior`

For custom Bayesian models, use `smooth_prior()` as a composable building block inside your own `@model`. Each call creates a sub-model that samples the smooth's parameters internally and returns the evaluated smooth function:

```
# Create smooth components (mixed-model reparameterization)
# Use bs=:tp to match the default in gam(@gam_formula(y ~ s(x, k=10)))
sm = GAM.gam_smooth(:x, dat; k = 10, bs = :tp)
```

```
SmoothMixedModel([0.664327533630193; 0.6628743574512967; ... ; -0.540699473296364; -
0.5426167225557019;;], [[0.15438623644410757 -0.2577943937355174 ... 0.03597281116708378 -
0.7659102718826961; 0.15440736781494313 -0.25766668968864265 ... 0.03788466790460092 -
0.7616640417613573; ... ; -0.13612135859607952 0.1606164144848799 ... -0.6484895009581195 -
0.03752349474514074; -0.13611571860612945 0.16045612470122486 ... -0.6500996527498536 -
0.04057732998841621]], [0.014406173358386995 -0.026837067992358504 ... -0.05154417204031194 6.1
16; 0.04344154074823492 0.004410031457064265 ... 0.5600267625508373 -5.399717439754989e-
15; ... ; 0.05008586618744659 0.012165648317958355 ... 0.5322891433528174 0.6649355842620508; 0.0
0.746900708784029], [0.10378107657020536, 0.11887176712494019, 0.17586602174257748, 0.216874
```

Smooth component:

```
Xf (null space): (200, 1)
Zs (penalized): [(200, 8)]
```

```
# Compose into a custom @model using to_submodel + prefix
```

```
@model function my_gam(y_obs, sm)
    0 ~ Normal(0, 10)
    ~ truncated(Normal(0, 2.5); lower = 0.0)

    # smooth_prior samples _f, _s, z internally; returns f(x)
    f ~ to_submodel(prefix(GAM.smooth_prior(sm), :s_x))

    y_obs ~ MvNormal(0 .+ f, ^2 * I)
end
```

```
custom_chains = sample(my_gam(dat.y, sm), NUTS(), MCMCThreads(), 2000, 2; progress = false)
```

Warning: Only a single thread available: MCMC chains are not sampled in parallel

@ AbstractMCMC ~/.julia/packages/AbstractMCMC/oqm6Y/src/sample.jl:544

Info: Found initial step size

= 0.05

Info: Found initial step size

= 0.025

Chains MCMC chain (2000×26×2 Array{Float64, 3}):


```

_gam = Matrix{Float64}(undef, n_draws_b, n_obs_c)
for i in 1:n_draws_b
    1_i = chains_b[Symbol(" [1]")].data[i]
    2_i = chains_b[Symbol(" [2]")].data[i]
    _s_i = chains_b[Symbol("_s[1]")].data[i]
    z_i = [chains_b[Symbol("z[$j]")].data[i] for j in 1:size(Zs_b, 2)]
    _gam[i, :] = X_b * [1_i] .+ Xf_b * [2_i] .+ _s_i .* (Zs_b * z_i)
end

# Sort by x
ord = sortperm(dat.x)
x_s = dat.x[ord]

# Posterior summaries
_custom = vec(mean(_custom; dims=1))[ord]
lo_custom = [quantile(_custom[:, j], 0.025) for j in 1:n_obs_c][ord]
hi_custom = [quantile(_custom[:, j], 0.975) for j in 1:n_obs_c][ord]

_gam = vec(mean(_gam; dims=1))[ord]
lo_gam = [quantile(_gam[:, j], 0.025) for j in 1:n_obs_c][ord]
hi_gam = [quantile(_gam[:, j], 0.975) for j in 1:n_obs_c][ord]

# --- Plots ---
p1 = scatter(dat.x, dat.y; alpha=0.2, color=:gray60, markersize=2, label="data",
    xlabel="x", ylabel="y", title="smooth_prior (composable API)")
plot!(p1, x_s, _custom; color=:steelblue, linewidth=2, label="posterior mean")
plot!(p1, x_s, lo_custom; fillrange=hi_custom, alpha=0.2, color=:steelblue,
    label="95% CI", linewidth=0)

p2 = scatter(dat.x, dat.y; alpha=0.2, color=:gray60, markersize=2, label="data",
    xlabel="x", ylabel="y", title="gam(..., priors=...) (brms-like API)")
plot!(p2, x_s, _gam; color=:darkorange, linewidth=2, label="posterior mean")
plot!(p2, x_s, lo_gam; fillrange=hi_gam, alpha=0.2, color=:darkorange,
    label="95% CI", linewidth=0)

# Scatter of posterior means
r_val = cor(_custom, _gam)
p3 = scatter(_gam, _custom; alpha=0.5, color=:purple, markersize=3,
    xlabel="gam() fitted", ylabel="smooth_prior fitted",
    title="Posterior mean correlation: r = $(round(r_val, digits=4))",
    aspect_ratio=:equal, legend=false)
mn = min(minimum(_gam), minimum(_custom))

```

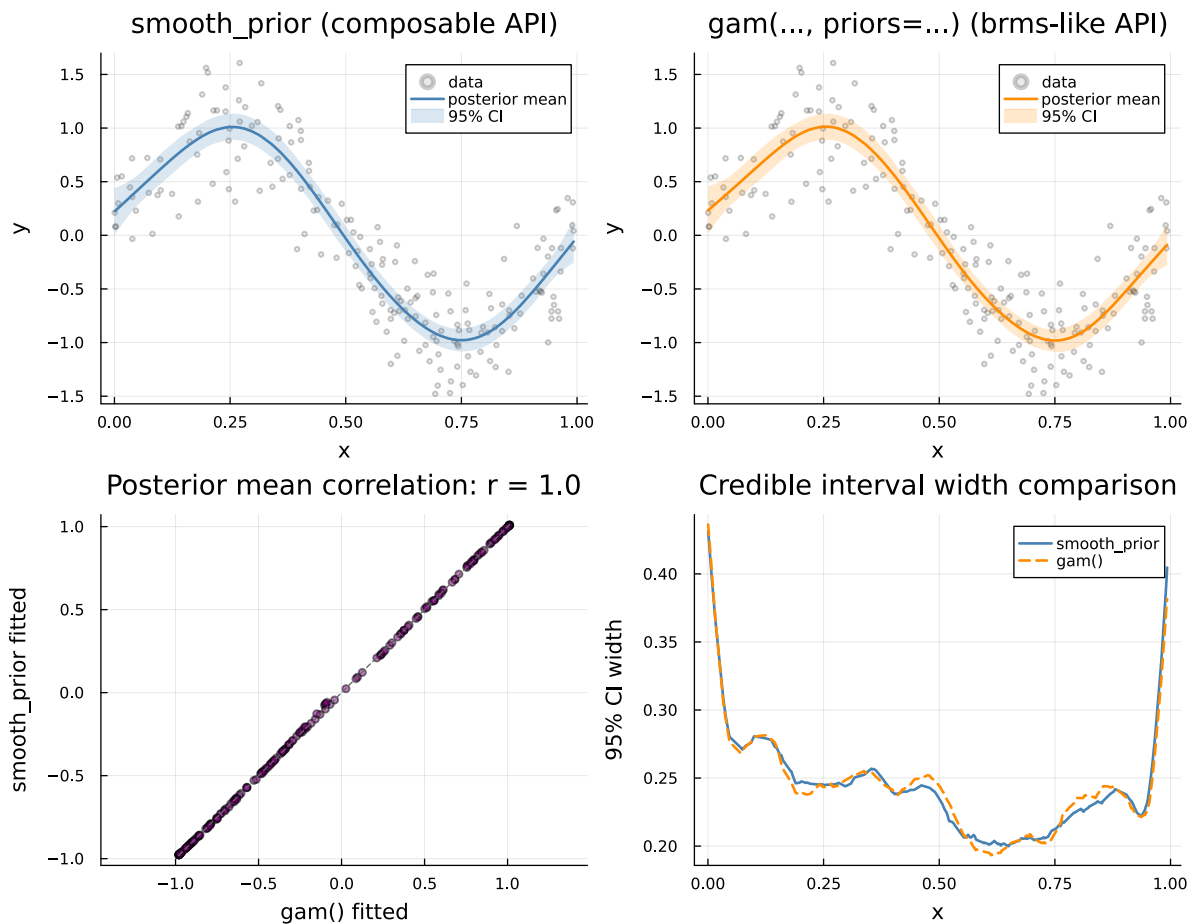
```

mx = max(maximum(_gam), maximum(_custom))
plot!(p3, [mn, mx], [mn, mx]; color=:gray40, linestyle=:dash, linewidth=1)

# CI width comparison
ci_w_custom = hi_custom .- lo_custom
ci_w_gam = hi_gam .- lo_gam
p4 = plot(x_s, ci_w_custom; color=:steelblue, linewidth=2, label="smooth_prior",
          xlabel="x", ylabel="95% CI width", title="Credible interval width comparison")
plot!(p4, x_s, ci_w_gam; color=:darkorange, linewidth=2, linestyle=:dash, label="gam()")

plot(p1, p2, p3, p4; layout=(2, 2), size=(900, 700))

```



```

Posterior mean correlation:  r = 1.0000
Max |mean difference|:      0.029586
Mean CI width (smooth_prior): 0.2409

```

Mean CI width (gam()): 0.2401

```
posterior: gam() mean=0.3154 sd=0.0160 | smooth_prior mean=0.3154 sd=0.0164
_s posterior: gam() mean=1.6161 sd=0.4276 | smooth_prior mean=1.6125 sd=0.4429
```

This is much cleaner than manually extracting matrices. For multiple smooths, give each a unique prefix:

```
sm1 = GAM.gam_smooth(:x1, data; k = 10)
sm2 = GAM.gam_smooth(:x2, data; k = 8, bs = :cr)

@model function multi_gam(y, sm1, sm2)
  0 ~ Normal(0, 10)
  ~ Exponential(1.0)
  f1 ~ to_submodel(prefix(GAM.smooth_prior(sm1), :s_x1))
  f2 ~ to_submodel(prefix(GAM.smooth_prior(sm2), :s_x2))
  y ~ MvNormal(0 .+ f1 .+ f2, ^2 * I)
end
```

Example 5: Low-level matrix extraction

For maximum control, extract the raw matrices with `gam_matrices()`:

```
# Extract matrices from a formula
gf = @gam_formula(y ~ s(x, k = 10))
X, sms, labels = GAM.gam_matrices(gf, dat)
```

```
([1.0; 1.0; ... ; 1.0; 1.0;;], SmoothMixedModel[SmoothMixedModel([0.664327533630193; 0.6628743
0.540699473296364; -0.5426167225557019;;], [[0.15438623644410757 -0.2577943937355174 ... 0.035
0.7659102718826961; 0.15440736781494313 -0.25766668968864265 ... 0.03788466790460092 -
0.7616640417613573; ... ; -0.13612135859607952 0.1606164144848799 ... -0.6484895009581195 -
0.03752349474514074; -0.13611571860612945 0.16045612470122486 ... -0.6500996527498536 -
0.04057732998841621]], [0.014406173358386995 -0.026837067992358504 ... -0.05154417204031194 6.
16; 0.04344154074823492 0.004410031457064265 ... 0.5600267625508373 -5.399717439754989e-
15; ... ; 0.05008586618744659 0.012165648317958355 ... 0.5322891433528174 0.6649355842620508; 0.
0.746900708784029], [0.10378107657020536, 0.11887176712494019, 0.17586602174257748, 0.216874
```

Fixed matrix X: (200, 1) (intercept)

Smooth 's(x,bs=tp)':

Xf (null space): (200, 1)

Zs (penalized): [(200, 8)]

Summary

Feature	Syntax
Default Bayesian GAM	<code>gam(formula, data; priors = PriorSpec())</code>
Custom priors	<code>PriorSpec(sds = Exponential(1.0), sigma = InverseGamma(2, 3))</code>
Per-smooth priors	<code>PriorSpec(specific = Dict("sds_s(x2)" => Exponential(0.5)))</code>
Poisson	<code>gam(formula, data; family = Poisson(), priors = PriorSpec())</code>
Composable smooth	<code>f ~ to_submodel(prefix(smooth_prior(sm), :name))</code>
Low-level matrices	<code>gam_matrices() + @model + sample()</code>
Coefficient table	<code>coeftable(m)</code> — posterior mean, SD, 95% CI
Credible intervals	<code>confint(m; level = 0.95)</code>
Full posterior	<code>m.chains[Symbol("_obs")]</code>

Key design choices

1. **Dispatch-based API:** The same `gam()` function handles both frequentist and Bayesian fitting — the presence of `priors=` triggers Bayesian mode via multiple dispatch.
2. **smooth2random decomposition:** Smooth terms are split into a fixed null-space (unpenalized, estimated via `fit`) and random-effects blocks (penalized, estimated via `smooth` × `z`).
3. **Composable smooth_prior:** Each smooth is a self-contained Turing `@model` that samples its own parameters. Compose multiple smooths via `to_submodel` with `prefix` to avoid name collisions.
4. **Non-centered parameterization:** Random effects use $\mathbf{b} = \sigma_s \cdot \mathbf{z}$ where $\mathbf{z} \sim N(0, I)$, which improves NUTS sampling geometry.
5. **Package extension:** Turing.jl is only loaded when needed (`using Turing`), keeping the base GAM.jl lightweight.